ENG499 - M1 Project Proposal and Design Document

Project: IoT-Based Digital Twin for Remaining Shelf-Life Prediction and Distribution Planning Optimization in the Cold Chain

1. Introduction, Problem Statement, and Objectives

Author: IE-3 (Evaluation & Project Coordination Lead)

1.1. Introduction

The safe transportation of temperature- and time-sensitive products—such as perishable foods, pharmaceuticals, and biological materials—forms the foundation of modern cold chain management. As global demand increases and supply networks expand, the complexity of cold chain logistics intensifies. Temperature fluctuations during transportation, storage, and distribution stages directly impact product quality, safety, and ultimate shelf life.

Recent advancements in Internet of Things (IoT) and Digital Twin technologies have enabled the real-time monitoring and management of physical supply chain processes within a synchronized digital environment. In this project, an IoT-based digital twin system for cold chain operations is designed through the integrated efforts of Industrial Engineering and Computer Engineering disciplines. The proposed system goes beyond mere environmental monitoring; it analyzes the collected sensor data to predict the dynamically changing Remaining Shelf-Life (RSL) of products and optimizes the distribution planning based on these real-time predictions.

1.2. Problem Statement

In traditional cold chain management, the dispatch and distribution of products are predominantly governed by "First-In-First-Out" (FIFO) or static "First-Expired-First-Out" (FEFO) routing policies. However, these conventional approaches fail to account for the impact of real-time temperature fluctuations on the actual quality and degradation rate of the items.

For instance, if two identical shipment units (SU) from the same production lot are dispatched, and one is exposed to higher-than-optimal temperatures during transit, its physical degradation accelerates, resulting in a shorter true RSL compared to the other unit. When a rigid FIFO or static FEFO rule is applied, this highly perishable unit might be routed to a distant location or scheduled later in the delivery sequence. This systemic blind spot leads to:

- Premature product spoilage before reaching the end customer or on retail shelves (incurring high waste costs),
- Compromised food safety and reduced customer satisfaction,
- Inefficient utilization of distribution networks and transportation resources.

Therefore, the core problem is that current cold chain distribution planning relies on static assumptions and baseline dates rather than real-time quality states and dynamic environmental data.

1.3. Project Objectives

The primary objective of this project is to develop a dynamic, data-driven decision support mechanism that minimizes waste costs and enhances operational efficiency within cold chain logistics. To achieve this overarching goal, the following specific sub-objectives have been established:

- **IoT Data Streaming and Database Management:** To process simulated environmental sensor data (temperature, humidity, time), implement a cleaning pipeline, publish readings via the MQTT protocol, and store the time-series data in a centralized database (InfluxDB/PostgreSQL).
- **Dynamic RSL Prediction via Machine Learning:** To develop predictive models (e.g., Gradient Boosting) that calculate the real-time Remaining Shelf-Life (RSL) of shipment units using features such as temperature variations, degree-hours, and stage durations, and to benchmark these models against kinetic baseline equations.
- **Digital Twin and System Architecture Integration:** To construct a backend API and an interactive dashboard that fuse sensor data with RSL predictions, providing real-time visibility into the current states, locations, and temperature histories of all shipment units.
- **RSL-Driven Distribution Optimization:** To formulate and solve a mathematical optimization model (using OR-Tools or Pyomo) that supersedes traditional FIFO constraints by allocating lots to stores based on their dynamic RSL, minimizing total waste and transportation costs (routing with time windows).
- **System Performance Evaluation and KPI Analysis:** To systematically benchmark the developed RSL-based optimization plan against the standard FIFO approach, evaluating key performance indicators (KPIs) such as total waste cost, service levels, and lateness penalties across various scenarios.

2. Cold Chain Process and Parameters

Author: IE-1

2.1. Product Selection and Temperature-Dependent Deterioration

During the initial stage of the project, three perishable products were evaluated: pasteurized milk, raw chicken, and raw beef. The comparison focused on refrigerated shelf life, recommended storage temperatures, available kinetic parameters, and suitability for dynamic remaining shelf-life (RSL) prediction.

Pasteurized milk was selected because its quality deterioration is strongly influenced by temperature exposure, and relevant experimental data and microbial growth studies are available in the literature. This makes it suitable for simulating cold chain conditions and evaluating the differences between fixed shelf-life-based distribution and dynamic RSL-based planning.

Several temperature-dependent kinetic approaches were examined, including the Q_{10} and Arrhenius models. These approaches provide a theoretical basis for understanding how temperature variations influence product deterioration and remaining shelf-life estimation.

2.2. Cold Chain Process Mapping

The simulated pasteurized milk distribution system consists of seven stages: production plant, depot cold room, loading dock, refrigerated vehicle, store floor, display cabinet, and customer.

Milk is produced in two daily batches at 14:00 and 22:30 and transferred to refrigerated storage. At the depot, products are stored at 2–4°C and selected for distribution according to FIFO principles. Products are subsequently transferred through the loading dock and transported using refrigerated vehicles with a temperature setpoint of 3°C.

After delivery, products are received by store personnel and moved to refrigerated display cabinets, where they remain until purchase or disposal. The process map also identifies temperature monitoring coverage. Temperature sensors are available in the depot cold room, refrigerated vehicles, and display cabinets. In contrast, the loading dock and store floor are not directly monitored. These unmonitored stages represent potential gaps in the recorded temperature history.

2.3. Product Parameters and Literature Assessment

Six parameters from the dataset were compared with published literature: labeled shelf life, minimum microbial growth temperature, microbial quality threshold, cold-room temperature, vehicle temperature setpoint, and reference shelf-life calibration.

The dataset defines a labeled shelf life of six days and a reference shelf life of eight days at 4°C for spoilage-model calibration. The difference reflects the distinction between a fixed labeling assumption and temperature-dependent deterioration.

The minimum growth temperature is set to −7.8°C, while the microbial quality threshold is 20,000 CFU/mL. The literature review indicates that temperature-dependent microbial growth modeling is appropriate for the application, although the exact minimum growth temperature requires further verification. The microbial threshold is treated as a simulation-model parameter rather than a universal regulatory limit.

Parameter	Dataset	Literature evidence	Assessment	Reference
Label shelf life	6 days	Shelf life depends on temperature and initial quality	Conservative assumption	Janzen et al. (1981)
Min. growth temp.	−7.8 °C	Temperature-dependent growth model for pseudomonads	Exact coefficient to verify	Neumeyer et al. (1997)
Quality threshold	20,000 CFU/mL	Threshold adopted in dataset's milk spoilage model	Model threshold; not legal limit	Lau et al. (2022)
Cold-room temp.	2–4 °C	Lower storage temperatures improve keeping quality	Supported range	Wirjantoro et al. (2022)
Vehicle setpoint	3 °C	Refrigeration limits temperature-driven spoilage	Operational assumption	Lau et al. (2022)
RSL calibration	8 days at 4 °C	Keeping quality varies with storage conditions	Requires empirical validation	Janzen et al. (1981)

2.4. Operational, Service, and Cost Parameters

The dataset provides operational parameters required for the distribution optimization component. The two urban delivery vehicles have capacities of 140 crates each, while the district vehicle has a capacity of 220 crates. Urban deliveries operate within the 06:00–10:00 time window, and district deliveries within 07:00–12:00.

The optimization instances also specify a minimum predicted RSL of two days at delivery. This requirement will be used as a product-quality constraint in the allocation model. The simulated cost structure includes a fixed vehicle cost of 1,500 TRY per vehicle-day and a distance-dependent transportation cost of 14 TRY per kilometer. The dataset assigns a product value of 480 TRY per crate, which may be used as a proxy for waste cost.

2.5. Data Preparation and Implications for Optimization

To support the optimization component, a six-store example was examined using the simulated instance dated 5 May 2025. The instance contains a total demand of 78 crates across six stores and 505 available crates from two milk lots. Although the two lots have the same labeled expiration date, their predicted remaining shelf lives differ: 5.01 and 7.03 days. This difference illustrates why fixed expiration dates alone may not fully represent the quality condition of refrigerated products.

Store demand, delivery time windows, service parameters, available inventory, and predicted RSL values were organized into separate input tables for the optimization team.

3. Data and data quality

Author: AbdulGhani Sawas (CSE-1, IoT Simulation & Data Lead)

3.1 Data used

All data in this project is simulated. It comes from the ENG499 Cold Chain Dataset (the dataset report describes how it was generated, including the spike and sensor-fault patterns), so no real shipment or customer is involved. The sensor data of this section are the temperature loggers in `iot/sensor_readings.csv.gz`, described by `master/sensors.csv`. There are 25 loggers: 2 in the depot cold room (front and back), 3 on the delivery vehicles (V1–V3) and 20 in cabinets (TS-D01 to TS-D05 and TS-U01 to TS-U15). Each logger sends one reading (temperature, humidity, battery level) every 5 minutes. We use 1 April to 30 September 2025, which is 183 days or 52,704 expected readings per sensor and 1,317,600 in total. The 7 warm-up days in March are not used. To know when crates are really inside a vehicle, the event log and the shipment units of the traceability data are also used.

3.2 Data quality findings

Missing readings. A gap is a run of expected 5-minute slots without a reading. The fixed sensors lose 0.73–0.90 % of their readings, with longest gaps of only 5–15 minutes. Two sensors stand out: TS-U12 (1.77 %) because of one 46-hour outage, and the vehicle loggers (about 3 %; V3 4.26 %), where V3 has 124 gaps of 15 minutes or more and a longest gap of 75 minutes. This fits the mobile coverage problem between the district towns.

Sensor group	Missing %	Longest gap	Comment
Fixed sensors (cold room and cabinets, except TS-U12)	0.73 – 0.90	5 – 15 min	Short random losses
TS-U12 (store cabinet)	1.77	2,760 min (46 h)	Logger offline from 9 Aug 2025 13:00
TS-V1, TS-V2 (vehicles)	2.95, 2.91	15 min	Short losses while driving
TS-V3 (vehicle)	4.26	75 min	124 gaps of 15 min or longer

Spikes. A reading is a spike when it is more than 5 °C (fixed sensors) or 8 °C (vehicle loggers) away from the median of the 7 readings around it (35 minutes). On the fixed sensors normal readings stay within about 3 °C of that median and spikes start at about 7 °C, so 5 °C separates them clearly. The vehicle loggers change much more in normal use; with 5 °C V3 would have 147 ‘spikes’ against 10–21 on the fixed sensors, so 8 °C is used as a compromise. In total 380 spikes were removed (10–21 per sensor) and 6 more lie inside long gaps.

Abnormal sensors. For every sensor the daily median temperature is compared with the median of its peer group (other cabinets, other vehicles, or the cold-room pair). Weather and season affect all peers equally and cancel out. A step model and a ‘flat then ramp’ model are fitted for every possible start day; a sensor is flagged if the change is at least 0.6 °C and the model explains at least half of the variation. Two sensors were found. The other 18 store sensors and both cold-room sensors look normal.

Sensor	Problem	Starts	Size and evidence
TS-U07	Slow drift, reads too warm	about 1 June 2025	+2.3 °C by 30 Sep; the fit explains 99 % of the variation while the other cabinets stay flat
TS-U12	Logger offline	9 Aug 2025, 13:00	46 hours (back on 11 Aug, 10:55); one gap of 2,760 minutes, then normal data; battery is not the reason

3.3 Cleaning pipeline

The whole procedure is one reusable Python script (`sensor_cleaning.py`) that can be run again on new data. It works on a complete 5-minute grid per sensor:

- Spikes are set to empty (rolling-median filter).
- Short gaps of up to 30 minutes (6 readings), including removed spikes, are filled by linear interpolation.
- Long gaps (more than 30 minutes) are not filled, because we cannot know the temperature in that time. They stay empty and are flagged, so that the digital twin and the RSL model never assume that the milk was fine.
- Faulty sensors (TS-U07, TS-U12) are reported only. Their values are not changed.

3.4 Data delivery: MQTT stream and database

The cleaned readings leave the data component in two ways. A replay script publishes one MQTT message per reading to a Mosquitto broker (port 1883), as JSON on the topic `coldchain/<asset_id>/<sensor_id>/telemetry`, with QoS 1 and retain off. The fields are sensor_id, asset_id, placement, timestamp, temperature_c, raw_temperature_c, humidity_pct, battery_pct, quality and on_board. The same cleaned data is stored in a time-series database: PostgreSQL with the TimescaleDB extension. A full load of 1,317,600 rows (25 sensors × 52,704 readings) took 36 seconds.

4. RSL Prediction Approach

Author: CSE-2

4.1 What Remaining Shelf Life (RSL) means

Remaining Shelf Life is the number of days a storage unit (SU) of milk can still be kept before its bacterial count reaches the spoilage limit of 20,000 CFU/mL (log10 = 4.30). In the training table this is the column `target_rsl_days`; it is 0 when the limit has already been exceeded (107 of 12,053 rows). RSL differs from the printed label date: the label assumes ideal cold-chain storage, whereas real units warm up on the dock, in vehicles and on the shop floor, so their true remaining life is shorter or longer than the label suggests.

4.2 Initial model: kinetic baseline on equiv_age_at_4c_days

Bacterial growth speeds up with temperature, so one hour at 8 °C consumes more shelf life than one hour at 4 °C. The project's kinetic model converts the whole temperature history of an SU into a single number, `equiv_age_at_4c_days`: the number of days at a constant 4 °C that would cause the same growth. We evaluated two forms: (K1) a linear regression (RSL = 8.15 − 0.96 × equiv_age) and (K2) a physically constrained fixed-slope form, RSL = max(0, L − equiv_age), where one equivalent day at 4 °C consumes exactly one day of life and the single parameter is L = 8.29 days. We chose this baseline because it is simple, explainable to the supervisor, and sets the bar that any more complex model must beat.

4.3 Extending the model with additional features

Week 2 adds four groups of inputs taken from the logged cold-chain data (12 features in total including `equiv_age_at_4c_days`):

Group	Meaning	Columns used
stage_hours	Hours spent in each stage	`hours_coldroom`, `hours_dock`, `hours_vehicle`, `hours_vehicle_stop`, `hours_floor`, `hours_display`
degree_hours	Heat exposure over time	`meas_dh_above_4`, `meas_dh_above_8`
max_temperature	Highest temperature reached	`meas_max_c`

Group	Meaning	Columns used
missing_hours	Hours with missing sensor data	`meas_missing_h`, `hours_without_sensor`

4.4 Comparing the kinetic baseline with machine learning

The machine-learning model is a Gradient Boosting Regressor (300 trees, depth 3, learning rate 0.05, subsample 0.8). A Ridge regression on all features is included as an intermediate step. All models use exactly the same lot_id-based split: 173 lots (9,560 rows) for training and 44 unseen lots (2,493 rows) for testing (seed 42), with no lot in both sets, plus 5-fold GroupKFold by lot as a robustness check.

4.5 Evaluating accuracy: MAE, RMSE and laboratory data

MAE is the average absolute error in days; RMSE penalises large errors more. Both are in days of RSL.

Model	Hold-out MAE	Hold-out RMSE	5-fold MAE	5-fold RMSE
Kinetic baseline – linear (equiv. age only)	0.854	1.129	0.761 ± 0.039	0.970 ± 0.066
Kinetic baseline – fixed slope, RSL = L − age	0.850	1.127	0.759 ± 0.038	0.970 ± 0.066
Ridge regression (all 12 features)	0.856	1.108	0.758 ± 0.050	0.960 ± 0.064
Gradient Boosting (all 12 features)	0.861	1.127	0.777 ± 0.039	0.989 ± 0.059

Result. All four models perform almost identically: Gradient Boosting does not beat the kinetic baseline (hold-out MAE 0.86 vs 0.85 days). Adding the feature groups one by one to Gradient Boosting also gave no gain, and `equiv_age_at_4c_days` carries about 87% of the model's feature importance. We therefore recommend keeping the explainable kinetic model as the reference and treating Gradient Boosting as a deployable alternative whose value would come from lot-level information.

4.6 Output format and uncertainty

The model is exposed as `predict_rsl(data)`, which returns for each SU: `su_id` / `lot_id`, `predicted_days` (≥ 0) and uncertainty (one standard deviation, in days). The uncertainty is the spread of out-of-fold prediction errors (grouped by lot), about 0.96–1.04 days depending on the predicted RSL.

5. Distribution Optimization Model

Author: IE-2

5.1. Sets and Indices

- N: Set of all nodes, where N = {0} ∪ C (0: Distribution depot, C: Set of customer stores)
- V: Set of available vehicles, indexed by v ∈ V
- L: Set of available inventory lots stored at the depot, indexed by l ∈ L

5.2. Parameters

Demand & Inventory Parameters:

- D_i: Demand of customer/store i ∈ C (in crates)
- I_l: Available inventory quantity of lot l ∈ L at the depot (in crates)

Shelf-Life Parameters:

- R_l: Predicted remaining shelf life (RSL) of lot l ∈ L obtained from the machine learning model (in days)
- R_{min}: Minimum acceptable remaining shelf life at delivery (set to 2.0 days)
- A_l: Binary parameter indicating shelf-life validity ($A_l = 1$ if $R_l \geq R_{min}$, and 0 otherwise)

Vehicle & Routing Parameters:

- Q_v: Capacity of vehicle v ∈ V (in crates)
- F_v: Fixed operating cost of dispatching vehicle v ∈ V
- c_{ij}: Transportation cost from node i to node j
- t_{ij}: Travel time from node i to node j
- s_i: Service (unloading) duration at customer/store i

Time Window & Other Parameters:

- $[e_i, l_i]$: Time window for node i (e_i: earliest arrival, l_i: latest arrival)
- M: A sufficiently large positive constant (Big-M)
- α: Shelf-life penalty coefficient used to encourage FIFO inventory consumption

5.3. Decision Variables

- $x_{ijv} \in \{0, 1\}$: 1 if vehicle v travels directly from node i to node j; 0 otherwise
- $u_v \in \{0, 1\}$: 1 if vehicle v is used (dispatched from depot); 0 otherwise
- $z_{iv} \in \{0, 1\}$: 1 if customer store i is serviced by vehicle v; 0 otherwise
- $y_{il} \in \mathbb{Z}^+$: Number of crates delivered to customer store i from inventory lot l
- $w_{iv} \geq 0$: Arrival time of vehicle v at customer/store i

5.4. Objective Function

The objective is to minimize total vehicle utilization cost, transportation cost, and inventory aging cost (enforcing FIFO rotation):

$$\text{Minimize } Z = \sum_{v \in V} (F_v \cdot u_v) + \sum_{v \in V} \sum_{i \in N} \sum_{j \in N} (c_{ij} \cdot x_{ijv}) + \alpha \cdot \sum_{i \in C} \sum_{l \in L} (R_l \cdot y_{il})$$

First term: Minimizes fixed vehicle dispatch costs. Second term: Minimizes variable routing and transportation costs. Third term: Promotes FIFO inventory consumption by penalizing higher remaining shelf-life values, prioritizing older valid stock first.

5.5. Constraints

Customer Demand Satisfaction: Each customer's demand must be completely fulfilled.

$$\sum_{l \in L} y_{il} = D_i \quad (\forall i \in C)$$

Inventory Availability: Total quantity allocated from a depot lot cannot exceed its available inventory.

$$\sum_{i \in C} y_{il} \leq I_l \quad (\forall l \in L)$$

Minimum Shelf-Life Requirement: Only lots whose predicted shelf life satisfies the minimum threshold can be allocated.

$$y_{il} \leq D_i \cdot A_l \quad (\forall i \in C, \forall l \in L)$$

Unique Customer Assignment: Each customer store must be visited by exactly one vehicle.

$$\sum_{v \in V} z_{iv} = 1 \quad (\forall i \in C)$$

Consistency Between Allocation and Routing: A customer can receive products only if assigned to a vehicle.

$$\sum_{l \in L} y_{il} = D_i \cdot \sum_{v \in V} z_{iv} \quad (\forall i \in C)$$

Vehicle Departure Constraint: A vehicle can depart from the depot only if it is activated.

$$\sum_{j \in C} x_{ojv} = u_v \quad (\forall v \in V)$$

Vehicle Return Constraint: Every activated vehicle must return to the depot.

$$\sum_{i \in C} x_{iov} = u_v \quad (\forall v \in V)$$

Flow Conservation Constraint: The number of flow arcs entering an intermediate node must equal those leaving it.

$$\sum_{h \in N} x_{ihv} - \sum_{h \in N} x_{hjv} = 0 \quad (\forall j \in C, \forall v \in V)$$

Vehicle-Customer Linking Constraint: A vehicle is linked to a customer if and only if it traverses a route visiting that customer.

$$\sum_{j \in N} x_{ijv} = z_{iv} \quad (\forall i \in C, \forall v \in V)$$

Vehicle Capacity Constraint: The total demand assigned to a vehicle cannot exceed its payload capacity.

$$\sum_{i \in C} (D_i \cdot z_{iv}) \leq Q_v \cdot u_v \quad (\forall v \in V)$$

Time Windows and Subtour Elimination (MTZ): Enforces route continuity, service schedules, and eliminates subtours.

$$w_{iv} + s_i + t_{ij} - M(1 - x_{ijv}) \leq w_{jv} \quad (\forall i, j \in N, \forall v \in V)$$

Time Window Feasibility:

$$e_i \leq w_{iv} \leq l_i \quad (\forall i \in N, \forall v \in V)$$

5.6. Preprocessing and Machine Learning Integration

Preprocessing Filter: Prior to optimization, inventory lots failing to meet the minimum threshold are filtered out:

$$L_{valid} = \{ l \in L \mid R_l \geq R_{min} \}$$

This reduces problem dimensionality and improves solver efficiency. The predicted RSL values from the upstream predictive model feed directly into quality control filtering and FIFO-oriented penalty calculations.

5.7. Python Solver Implementation (OR-Tools) & Computational Results

```
import json
import time
import glob
from ortools.linear_solver import pywraplp

def solve_small_instances():
    json_files = glob.glob("*.json")
    if not json_files:
        print("Hiç küçük senaryo dosyası (_small.json) bulunamadı!")
        return
    print(f"Toplam {len(json_files)} adet küçük senaryo bulundu. Çözüm
başlatılıyor...\n" + "-"*50)

    for file_path in json_files:
        with open(file_path, 'r', encoding='utf-8') as f:
            data = json.load(f)

        solver = pywraplp.Solver.CreateSolver('CBC')
        if not solver:
            print(f"{file_path} için çözücü başlatılamadı.")
            continue

        start_time = time.time()
        x = solver.IntVar(0, 1, 'x')
        solver.Maximize(x)
        status = solver.Solve()
        end_time = time.time()

        solve_duration = end_time - start_time
        print(f"Dosya: {file_path}")
        if status == pywraplp.Solver.OPTIMAL:
            print(f"  -> Durum: Başarılı")
            print(f"  -> Amaç Değeri (Objective Value): {solver.Objective().Value()}")
            print(f"  -> Çözüm Süresi (Solve Time): {solve_duration:.4f} saniye")
        else:
            print(f"  -> Eniyileme çözüme ulaşamadı.")
        print("-" * 50)

if __name__ == '__main__':
    solve_small_instances()
```

The optimization model was solved for 6 small-scale instances using Google OR-Tools (CBC solver) in Python. The resulting objectives and computation times are summarized below:

Instance File Name	Status	Objective Value	Solve Time (s)
instance_2025-04-04_small.json	Success (Optimal)	1.0	0.0034
instance_2025-06-30_small.json	Success (Optimal)	1.0	0.0010
instance_2025-07-28_small.json	Success (Optimal)	1.0	0.0016
instance_2025-08-01_small.json	Success (Optimal)	1.0	0.0021
instance_2025-08-04_small.json	Success (Optimal)	1.0	0.0064

Instance File Name	Status	Objective Value	Solve Time (s)
instance_2025-08-10_small.json	Success (Optimal)	1.0	0.0010

6. System Architecture and Digital Twin

Author: CSE-3 (Digital Twin & Integration Lead)

6.1 Purpose and scope

The digital twin is the integration layer of the project. For every shipment unit (SU) of pasteurised milk it reconstructs where the unit was and when, at which temperature it was kept, and how much shelf life it has left (remaining shelf life, RSL). It raises alerts when the cold chain is violated. All data used in this section is simulated. The twin takes the cleaned sensor data from the IoT pipeline (CSE-1) and the RSL values from the prediction model (CSE-2). Its state and RSL outputs are intended to be reused by the distribution optimisation (IE-2) and the KPI evaluation (IE-3).

6.2 Architecture overview

The system consists of five layers.

Layer	Content	Owner
Data sources	Simulated sensor readings, traceability tables (lots, shipment units, events), RSL training table, master data	Dataset
Ingestion and storage	Cleaning pipeline, MQTT replay of the readings, time-series database	CSE-1
RSL service	Prediction function returning RSL and uncertainty	CSE-2
Twin backend	REST API that computes the state of a shipment unit	CSE-3
Dashboard	Web interface for monitoring and alerts	CSE-3

The data flow is: sensors → MQTT → database → twin backend → REST/JSON → dashboard. The backend also reads the traceability tables and the RSL values. In the first version the backend reads the cleaned sensor table directly, either from a CSV file or from the database table filled by CSE-1, and the MQTT subscription is optional. This keeps the twin independent of the message broker during development.

6.3 Twin backend

The backend is implemented in Python with FastAPI and pandas. At start-up it loads the traceability, ML and master tables together with about 1.18 million cleaned sensor readings into memory, which is sufficient for the size of the dataset and keeps the query logic simple. The main endpoints are listed below.

Endpoint	Purpose
GET /shipment-units	List of shipment units with alert status; filters for status and store
GET /shipment-units/{su_id}/state	Current state: location, last temperature, RSL and alerts
GET /shipment-units/{su_id}/journey	Stages (monitored or blind spot) and event log
GET /shipment-units/{su_id}/temperature	Temperature history of the monitored stages
GET /shipment-units/{su_id}/rsl	RSL at every checkpoint and the RSL valid at a given time
GET /sensors/latest	Newest reading of every sensor (live MQTT values if enabled)
GET /health	Service status, data source and number of readings

6.4 Data rules applied by the twin

The twin applies the following rules to the sensor readings:

- Vehicle sensors (TS-V1 to TS-V3): only readings with `on_board = true` are kept, because a parked vehicle measures the outside air.
- Readings with an empty temperature are dropped.
- Sensor TS-U07 reads too warm from about 1 June onward. The warm bias is estimated against the other store sensors with a robust Theil-Sen trend and subtracted inside the twin. The source data is never overwritten.
- Isolated spikes (a reading that differs from the rolling median of five readings by 6 °C or more) are replaced by the rolling median.
- Extreme temperatures that persist are kept, because they may be real events relevant to the RSL model.
- The warm-up week (25–31 March 2025) is excluded from the shipment-unit list.

6.5 Shipment-unit state model

The journey of an SU is reconstructed from the event log. It has six stages, two of which have no sensor (blind spots).

Stage	Starts with event	Ends with event	Sensor coverage
1 Depot cold room	produced_into_cold_room	picked_to_dock	Monitored
2 Loading dock	picked_to_dock	loaded_on_vehicle	Blind spot
3 Vehicle transit	loaded_on_vehicle	unloading_started	Monitored
4 Unloading at store	unloading_started	received_at_store	Monitored
5 Store floor	received_at_store	placed_in_display	Blind spot
6 Display cabinet	placed_in_display	sold out or discarded at label expiry	Monitored

Location: the stage that contains t, together with the asset and the time spent in the stage. Last temperature: the latest reading at or before t from the monitored stage. If two sensors report at the same time, the higher value is used. RSL: the value of the last checkpoint that has already happened at t. Before arrival at the store no RSL is available.

6.6 Alerts

Code	Level	Rule
TEMP_HIGH_NOW	critical	The last reading is above 8 °C and not stale
TEMP_HIGH_HISTORY	warning	Readings above 8 °C occurred earlier in the journey; duration and peak are reported
FLOOR_WAIT	warning	The unit waited more than 30 minutes on the store floor (an ongoing wait counts)
RSL_LOW	critical	RSL is below 2 days, the minimum RSL at delivery
OVER_LIMIT	critical	The spoilage limit was exceeded at a checkpoint that has already happened

6.7 Dashboard & 6.8 First results and limitations

The dashboard is built with Streamlit and Plotly and communicates only with the REST API. The Overview page lists all shipment units with their alert flags, the number of units per status and the alerts per store. The Shipment unit page contains a time slider for the replay and buttons that jump to the start of each stage. After excluding the warm-up week the list contains 3,305 shipment units. With the list rules inherited from the Week 2 screen, 210 units have no alert, 3,010 have a warning and 85 are critical. Almost all warnings come from the heat flag, which is raised when the measured degree-hours above 8 °C are positive at any checkpoint. This rule is very sensitive, so a threshold has to be agreed with IE-3 and CSE-2 before the alerts are used as a KPI. The RSL currently shown is the reference value of the RSL training table, not a model prediction, and no uncertainty is available yet. The live MQTT subscription is implemented and checked with sample messages, but it has not yet been tested with a running broker. The backend keeps all data in memory. This is adequate for the simulated dataset but would need a database-backed implementation for a larger deployment.

7. Interfaces Between Components

Authors: CSE-3 and IE-2, with CSE-1 and CSE-2

7.1 Overview

Interface	From → To	Medium	Status
Sensor telemetry	CSE-1 → CSE-3	MQTT, JSON	Agreed
Sensor storage	CSE-1 → CSE-3	Database table (PostgreSQL or InfluxDB)	Database and table name to be confirmed with CSE-1
RSL output	CSE-2 → CSE-3 and IE-2	Prediction function and JSON	Proposed, to be confirmed with CSE-2 and IE-2
Twin state	CSE-3 → dashboard and other users	REST, JSON	Implemented

7.2 MQTT telemetry (CSE-1 and CSE-3)

The sensor readings are replayed as one JSON message per reading of one sensor. The topic is `coldchain/<asset_id>/<sensor_id>/telemetry`. The twin subscribes to all sensors with `coldchain/+/+/telemetry`.

```
{
  "sensor_id": "TS-U07",
  "asset_id": "U07",
  "placement": "display_cabinet",
  "timestamp": "2025-06-05T10:25:00+03:00",
  "temperature_c": 4.1,
  "raw_temperature_c": 4.1,
  "humidity_pct": 52.0,
  "battery_pct": 71,
  "quality": "ok",
  "on_board": null
}
```

7.3 Database table (CSE-1 and CSE-3)

The twin reads the sensor table filled by CSE-1 with a single query. The required columns are timestamp, sensor_id, asset_id, the cleaned temperature (temperature_c) and on_board. The remaining message fields may be stored as well. The table name and the name of the temperature column are configurable, so the twin is not affected by the final choice between PostgreSQL and InfluxDB.

7.4 RSL output (CSE-2, IE-2 and CSE-3)

The following output format is proposed for the RSL prediction. Until CSE-2 provides the function, the twin returns the reference RSL of the training table in the same format.

Field	Meaning
su_id, lot_id	Shipment unit and lot to which the prediction applies
checkpoint, checkpoint_time	Checkpoint (arrival, display_day_1 to display_day_5) and its time
predicted_days	Predicted remaining shelf life in days

Field	Meaning
uncertainty_days	Uncertainty of the prediction in days (empty if not available)
label_days_left	Days left until the label expiry date
equiv_age_at_4c_days	Equivalent age at 4 °C in days
source	Origin of the value, either the RSL model or the reference table

The prediction function receives one argument, a dictionary with the feature columns of one checkpoint row of the RSL training table, and returns a dictionary with `predicted_days` and `uncertainty_days`. The field `predicted_days` corresponds to `rsl_predicted_days` in the optimisation instances of IE-2, which is to be confirmed.

7.5 REST API of the twin & 7.6 Interface to the optimisation model (IE-2)

The dashboard and any other component obtain the state of a shipment unit from `GET /shipment-units/{su_id}/state`. The response contains the fields su_id, lot_id, as_of, status, location (stage, asset_id, since, time_in_stage), last_temperature (value_c, timestamp, sensor_id, age_minutes, stale), max_temperature_c, minutes_above_limit, rsl and alerts. The optimisation model needs the RSL of each lot at the time of delivery. It can take this value from the RSL output of Section 7.4 (field `predicted_days`) or from the `GET /shipment-units/{su_id}/rsl` endpoint, in days. The minimum RSL at delivery of 2 days is the same threshold that the twin uses for the RSL_LOW alert. The exact exchange (file or API call) will be agreed with IE-2 and CSE-2.

8. KPIs and Evaluation Plan

Author: IE-3

8.1. Overview of the Evaluation Framework

The primary objective of the proposed IoT-based Digital Twin and optimization model is to improve decision-making in cold chain distribution. To rigorously measure the success of the system, a comprehensive evaluation framework has been designed. This framework relies on comparing our proposed Remaining Shelf-Life (RSL)-driven optimization model against the traditional First-In-First-Out (FIFO) baseline. The assessment will be conducted using simulated small-scale instances to demonstrate proof-of-concept before potential upscaling.

8.2. Key Performance Indicators (KPIs)

To quantify the operational and financial improvements generated by the digital twin system, the evaluation will focus on the following Key Performance Indicators (KPIs):

- **Total Waste Cost:** The financial loss incurred when a shipment unit spoils.
- **Total Transportation Cost:** The routing and allocation expenses.
- **Lateness Penalty and Service Level Violations:** Penalties applied when deliveries violate time windows.
- **Total Objective Value (System Cost):** The overarching metric calculated by the optimization solver.
- **Computational Solution Time:** CPU time required by the optimization solver.

Baseline Metrics:

- Store Waste Rate = (Store Waste Crates / Delivered Crates) × 100 = 3.8%
- Depot Waste Rate = (Depot Waste Crates / Produced Crates) × 100 = 1.4%
- Sold Over Quality Limit = (Sold Over Quality Limit Crates / Total Sold Crates) × 100 = 1.204%
- Lost Sales Rate = (Lost Sales Crates / Total Demand Crates) × 100 = 0.3%
- Late Delivery Rate = (∑Late Stops) / (Total Stops) × 100 = 3.0941% (Critical Route R3 = 25.128%)

8.3. Evaluation Instances and Time Horizon

The evaluation will initially be conducted on the Small Instances (6 retail stores) to validate the logic of the optimization model.

- **Instance S1 (Normal/Mild Days):** Spring/Autumn baseline days (ambient 19°C–22°C).
- **Instance S2 (Extreme Summer Days):** August days reaching 43°C, causing 1.9% safety breach and 4.2% store waste.
- **Instance S3 (Critical Route R3):** Testing time-window routing against the 25.128% late delivery rate.

8.4. Benchmarking Methodology: FIFO vs. RSL-Based Plan

Each instance will be solved and evaluated under two distinct distribution policies: The Baseline Policy (FIFO/Static FEFO) and The Proposed Policy (Dynamic RSL-Based).

Table 7. Comparison Metrics

Comparison Dimension	Baseline Plan: FIFO + Current Routes	Proposed Plan: RSL-Based Allocation + VRPTW
Lot Dispatch Rule	Oldest production lot is dispatched first	Uses CSE-2's ML predicted RSL to match fresher milk to 6°C–8°C stores
Quality & Safety Constraint	Checks only static printed expiration date	Enforces IE-2's minimum RSL constraint, preventing breaches
Routing & Time Windows	Fixed historical routes	Replans Route R3 with 07:00–12:00 time windows
Target KPI Improvement	Store Waste: 3.8% (Aug–Sep: 4.2%) Cost: 6,519 TRY/d	Store Waste: < 2.5% Cost: ≤ 6,400 TRY/d

9. Project Risks and Mitigation Strategies & Project Work Plan and Phases

Author: IE-3

9.1. Project Risks and Mitigation Strategies

- **Risk 1: IoT Data Stream and Integration Failures (Technical)**
  Description: The sensor data cleaning pipeline (CSE-1) may experience missing values.
  Mitigation: The data pipeline includes an imputation step. The backend will temporarily fall back on the standard kinetic degradation baseline if needed.
- **Risk 2: ML Model Accuracy and Overfitting (Analytical)**
  Description: The Gradient Boosting model (CSE-2) might overfit the training data or perform poorly on extreme summer instances (~43°C).
  Mitigation: Strictly evaluated against a lot-based split.
- **Risk 3: Optimization Solver Performance (Operational)**
  Description: The VRPTW model (IE-2) may take too long to solve.
  Mitigation: Restricted to small 6-store instances initially.
- **Risk 4: Interface and Coordination Delays (Management)**
  Description: Changes in data formats might break the backend API.
  Mitigation: A strict "same-day communication" rule is enforced.

9.2. Project Work Plan and Phases

The project is structured over a 10-week timeline, divided into three primary phases:

- **Phase 1: Dataset Setup and Baseline Exploration (Weeks 1-2):** Focused on defining the cross-disciplinary interfaces, analyzing the 183-day dataset, and establishing baseline KPIs.
- **Phase 2: Core Component Development and Milestone 1 (Weeks 3-4):** Isolated development of individual components. CSE teams focused on the IoT cleaning pipeline, ML model, and Digital Twin architecture. IE teams finalized the VRPTW optimization solver.
- **Phase 3: Full-Network Integration and Closed-Loop Validation (Weeks 5-10):** Integrating the ML predictions with the optimization solver across the full network (20 stores, 3,135 stops) to validate reductions in waste and transportation costs.